



Universidad Nacional de Salta

Facultad de Ciencias Exactas

Licenciatura en Análisis de Sistemas

Algoritmos y Estructuras de Datos

**Recorridos: El "Matching Engine" de UBER - El Problema de Envío de Unidades
cálculo del ETA.**

Grupo N° 5

Contino Sarmiento, Lucas Joaquín

Salasar, Josué Mateo

Villagra, Carlos Emanuel

Diego Rodriguez

Gerardo Cabero

Carlos Nocera

14 de junio del 2026

Introducción

El presente trabajo consiste en el desarrollo de un simulador de asignación de vehículos basado en aplicaciones de transporte urbano como puede ser UBER. El sistema modela el proceso mediante el cual un conjunto de usuarios solicita viajes desde diferentes puntos de la ciudad de Salta, mientras un grupo de vehículos disponibles es evaluado para determinar cuál puede atender la solicitud en el menor tiempo posible.

Para ello se emplean estructuras de datos lineales y no lineales, grafos dirigidos ponderados, colas de prioridad y algoritmos de caminos mínimos.

Objetivos

Objetivo general

El objetivo de este proyecto es implementar un sistema de gestión de envío de unidades de APP para Grande Ciudades mediante el desarrollo de un simulador de asignación de vehículos utilizando grafos y estructuras de datos vistas en la materia.

Objetivos específicos

- Representar el mapa de Salta mediante un grafo dirigido.
- Calcular rutas mínimas entre usuarios y vehículos.
- Implementar una cola de prioridad para la asignación de taxis.
- Simular aceptación o rechazo de viajes por parte de los conductores.
- Visualizar el proceso mediante una interfaz gráfica.

Descripción general de la solución implementada

La solución desarrollada consiste en un simulador de asignación de vehículos inspirado en aplicaciones de transporte urbano. El sistema representa el mapa de la ciudad de Salta mediante un grafo dirigido, donde cada esquina corresponde a un vértice y las calles corresponden a aristas ponderadas.

Los datos geográficos fueron obtenidos a partir de archivos CSV y GeoJSON que contienen información sobre coordenadas, intersecciones y tipos de calles.

Cuando un usuario solicita un viaje, el sistema calcula el tiempo estimado de llegada (ETA) de cada vehículo disponible utilizando distancias geográficas y velocidades asociadas al tipo de vía.

Posteriormente, los vehículos son ordenados en una cola de prioridad según su ETA, permitiendo seleccionar la unidad más conveniente. Finalmente, el recorrido es visualizado mediante una interfaz gráfica desarrollada en JavaFX, donde se anima el desplazamiento del taxi siguiendo el camino mínimo calculado por el algoritmo de Dijkstra.

Análisis del problema

Las reglas que impusimos como base antes de empezar a desarrollar una solución a nuestro problema son:

Usuarios:

- Solo pueden solicitar viajes desde una esquina.
- Las solicitudes de viaje se almacenan en una lista.

Unidades:

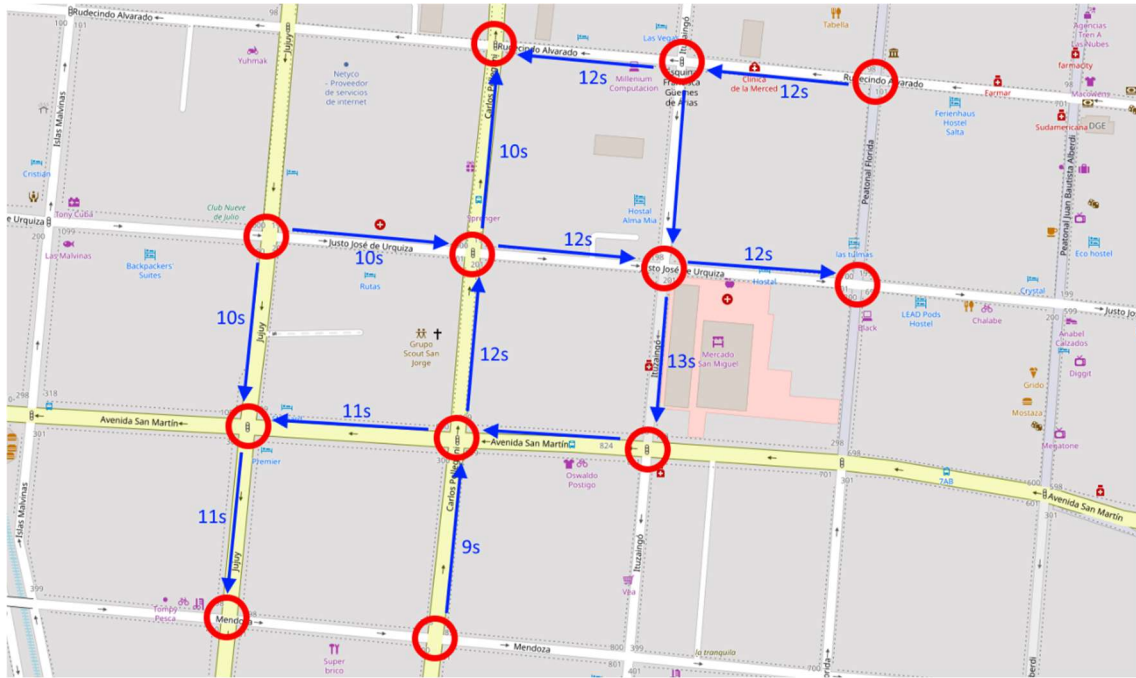
- Se ubican inicialmente en esquinas al azar.
- Pueden encontrarse disponibles u ocupadas.

Asignación:

- Las unidades disponibles son ordenadas según su ETA en una cola de prioridad.
- Las unidades disponibles deben tener la posibilidad de rechazar un viaje.
- Si una unidad rechaza el viaje, se intentará una asignación con la siguiente unidad en cola.

Modelado del mapa

Con el objetivo de llevar el mapa de Salta a una forma donde la podamos trabajar en código representamos todas las intersecciones de calles, es decir las esquinas, como un vértice en un grafo y cada calle como una arista dirigida donde el peso de estas aristas representa el tiempo estimado de recorrido.



Los valores mostrados son de ejemplo y no corresponden a datos reales.

Estructuras de datos utilizadas

Para almacenar la información del mapa hicimos uso de:

- Matriz de adyacencia (no ponderada) para almacenar las esquinas.
- Matriz de adyacencia ponderada para almacenar el tiempo en que se recorren los caminos.
- Cola de prioridad para ordenar las unidades según su ETA.
- Listas para almacenar usuarios, unidades disponibles y guardar rutas

Justificación de las estructuras de datos utilizadas

- **Matriz de adyacencia:** fue elegida debido a que el número de vértices del grafo es conocido. Permite consultar en rápidamente si existe una conexión entre dos esquinas.
- **Matriz ponderada:** se utilizó para almacenar el costo asociado a cada arista (que representa una calle), representado por el tiempo estimado de recorrido entre esquinas.
- Cola de prioridad: fue seleccionada porque permite obtener eficientemente el vehículo con menor ETA, simulando el proceso de asignación utilizado por aplicaciones reales de transporte.
- **Listas dinámicas:** fueron utilizadas para almacenar usuarios, taxis y rutas debido a la facilidad para insertar y eliminar elementos durante la simulación.

- **Mapas o diccionarios:** se emplearon para asociar identificadores de nodos con información geográfica y nombres de esquinas, permitiendo búsquedas rápidas.

Uso de las estructuras dentro de la aplicación

El grafo dirigido representa la red de calles de la ciudad de Salta. Cada vértice corresponde a una esquina y cada arista representa una calle.

Las matrices de adyacencia almacenan las conexiones y los costos de desplazamiento entre nodos.

Los usuarios y vehículos son generados en nodos aleatorios del grafo y almacenados en listas dinámicas.

Cuando un usuario solicita un viaje, se calcula el ETA de todos los taxis disponibles. Estos valores son insertados en una cola de prioridad, permitiendo obtener el vehículo con el menor tiempo de llegada.

Finalmente, la ruta mínima calculada por Dijkstra es utilizada por la interfaz gráfica para animar el movimiento del vehículo sobre el mapa.

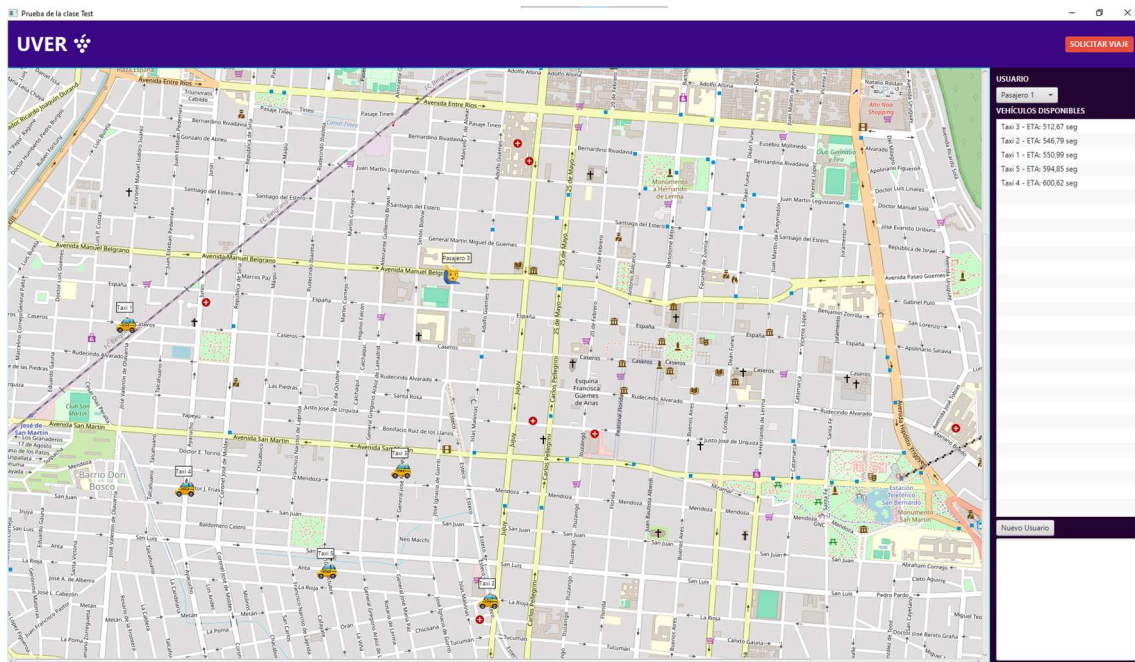
Algoritmos implementados

- **Fórmula de Haversine:** utilizada para calcular la distancia más corta entre dos puntos sobre la superficie de una esfera, utilizando únicamente sus coordenadas de latitud y longitud.
- **Algoritmo de Dijkstra:** utilizado para encontrar la ruta más corta desde un nodo de origen hacia todos los demás nodos en la matriz de adyacencia ponderada.

Diseño de la simulación

- 1) Se carga el mapa de Salta en un grafo dirigido.
- 2) Se generan unidades en esquinas aleatorias del mapa.
- 3) Se generan usuarios en esquinas aleatorias del mapa.
- 4) Un usuario solicita un viaje.
- 5) Se calcula el ETA para cada unidad.
- 6) Se crea una cola de prioridad en base a los ETA obtenidos.
- 7) Se asigna una unidad al usuario.
- 8) Se anima el recorrido en pantalla.

Interfaz gráfica



Componentes:

- Mapa de Salta.
- Lista desplegable que contiene los usuarios.
- Lista de unidades ordenados por ETA.
- Botón de solicitud de viaje.

Resultados obtenidos

El programa logró representar aproximadamente 1665 esquinas de la ciudad de Salta mediante un grafo dirigido. Las rutas mínimas fueron calculadas correctamente mediante el algoritmo de Dijkstra y la asignación de vehículos se realizó utilizando una cola de prioridad basada en ETA.

Conclusiones

El proyecto permitió aplicar de manera práctica conceptos fundamentales de la materia, tales como grafos, listas, matrices, colas de prioridad y algoritmos de caminos mínimos. Asimismo, la incorporación de una interfaz gráfica facilitó la visualización del funcionamiento interno de las estructuras de datos y los algoritmos implementados.